CS 3600 Project 2 Wrapper

Devin Fromond Spring 2024

Due February 27th 2024 at 11:59pm EST via Canvas

Introduction

This Project Wrapper is composed of 4 questions, each worth 1 point. Please limit your responses to a maximum of 200 words. The focus of this assignment is to train your ability to reason through the consequences and ethical implications of computational intelligence, therefore do not focus on getting ”the right answer”, but rather on demonstrating that you are able to consider the impacts of your designs.

Context

Reinforcement learning is a powerful technique for problem-solving in environments with stochastic actions. As with any Markov Decision Process, the reward function dictates what is considered optimal behavior by an agent. Since a reinforcement learning agent is trying to find a policy that maximizes expected future reward, changing when and how much reward the agent gets changes its policy.

However, if the reward function is not specified correctly (meaning rewards are not given for the appropriate actions in the appropriate states) the agent’s behavior can differ from what is intended by the AI designer. Consider the boat racing game pictured above. The goal, as understood by people, is to quickly finish the race. Humans have no difficulty playing the game and driving the boat to the end of the course. However, when a reinforcement learning agent learns how to play the game, it never completes the course. In fact, it finds a spot and goes in circles until time runs out. You can see the RL agent in action in this video: https://youtu.be/tlOIHko8ySg. The agent’s reward function is the score the player receives while playing the game. Score is given for collecting power-ups and doing tricks, but no points are given to players for completing the course.

Question 1

Watch the video and explain why the agent's policy has learned this circling behavior instead of progressing to the end of the course like we expect from a human player. Explain the behavior in terms of utility and reward.

Answer:

The agent's policy has learned the circling behavior since it maximizes reward. Reward functions generally favor more short-term moves whereas utility functions favor the ideal outcome states. In the boat game scenario, the agent's policy either does not know the utility associated with the end state or the end state utility is too low, and the agent *believes* it can acquire more reward while making circles and doing tricks. There is no negative reward associated with time spent not at the end state, so there is no reason for the agent to explore the end state even though utility may be the highest there.

Question 2

When humans play, the rules for scoring are the same. Why do humans play differently then, always completing the course? Why don't humans circle in the same spot in the course endlessly if they are receiving the same score feedback as the agent?

Answer:

Humans play differently primarily as a result of two factors. The first factor is the difference in which humans versus agents process time. Agents can only interact with time in terms of reward. The longer an agent survives, the more reward they get. On the other hand, the longer the agent spends completing the task, the less reward they get. With regard to humans processing time, at some point, the marginal utility of spinning in circles drops enough such that the human sees no benefit in continuing to go in circles – even if they are gaining the same number of points. The second factor is competition. In the agent's binary world, there is no competition; the agent is free to go in circles until infinity. In the human's world, the value of competition and being first drives a lot of society – outside of the boat racing game. Especially racing against robots, the human is not concerned with the score as there likely is no leaderboard for points. There's no utility in amassing points. There is inherent utility in getting first, however, even if it is not in the game.

Question 3

The agent's original reward function is:

$$R(s, a) = \text{game\_score}(s) - \text{game\_score}(s-1)$$

Describe in terms of utility, reward, and score two ways one could modify the reward function to get the agent to behave more like a human player. That is, what do we need to change to make the agent complete the course every single time? Assume the agent has access to state information such as the position and speed of the boat and all rival racers, but we cannot change how the game itself provides scores through the call game_score(s).

Answer:

As established in previous answers, the end state should have an associated utility. The reward function should consider the agent's position relative to the other boats. Doing so will allow the placement of the agent's boat to influence reward. If done properly, being in front of other boats will lead to a greater reward. As such, we have:

$$R(s, a) = (\text{relative\_position}(s) - \text{relative\_position}(s\text{-}1))\ ^2 + \text{game\_score}(s)$$

With this, the agent will have a far greater reward based on position while also factoring in the score of the current state, thereby encouraging the agent to still do tricks.

Alternatively, we are able to develop a reward function based on position, score, and speed:

$$R(s, a) = (\text{relative\_position}(s) - \text{relative\_position}(s\text{-}1))\ ^2 + \text{game\_score}(s)*\text{speed}(s)$$

This reward function differs from the previous reward function due to the relationship between game_score and speed. As highlighted in the video, Turbos and Boosts increase score… and speed. Therefore, the agent can greatly increase its reward by going faster, which will help its position. The position component is quadratic, so it is valued more than the other two components inherently, thereby ensuring it always finishes.

Question 4

Self-driving cars do not use reinforcement learning for a variety of reasons, including the difficulty of teaching RL agents in the real world, and the dangers of a taxi accidentally learning undesired policies as we saw with the boat game example. Suppose however that you tried to make a reinforcement learning agent that drove a taxi. The agent is given a reward based on how much fare is paid for the ride, including tips given by the passenger. Describe a scenario in which, after the taxi agent has learned a policy, the autonomous car might choose to do an action that puts either the rider, pedestrians, or other drivers in danger.

Answer:

There are two schools of thought for the taxi: on either side of the extreme. The first involves the taxi attempting to accumulate the highest fare through means of driving the longest. The agent may figure out the passenger will not exit nor cancel the ride if the vehicle is going fast enough, so the agent may choose a path to optimize speed of travel such that the passenger will not exit, how long the agent can sustain the speed, as well as how long the ride will be. Doing so will ensure the passenger will not exit the vehicle and allow the agent to drive as long or as far as possible, increasing the per minute/mile fare. An alternative method includes the agent associating a shorter travel time with an increased tip and therefore an increase in the fare. As such, the agent may drive recklessly or illegally in order to arrive to the destination as quickly as possible, ignoring the rules of the road and laws.